

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Jeeva	Reg. No.:	192421334
Section / Batch:	Information Technology	Date:	25/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q31. Clique Detection Problem (NP-Complete / Backtracking Problem)

Given a graph, determine whether a clique of size k exists. Clearly define constraints such as vertex adjacency and clique size requirement. Analyze how constraints affect the combinatorial search space. Develop a backtracking-based approach to generate subsets of vertices. Apply pruning techniques to discard non-clique candidates. Justify how your solution solves the problem and discuss its NP-Complete nature.

Code of Conduct

- I Jeeva (192421334) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date:

Q31. Clique Detection Problem — Backtracking-Based Constraint Solving

1. Problem Definition

The Clique Detection Problem is a classical combinatorial decision problem on graphs. Given:

- An undirected graph $G = (V, E)$
- A positive integer k (the required clique size)

The objective is to determine whether there exists a subset $S \subseteq V$ of size k such that every pair of vertices in S is connected by an edge in E — that is, S forms a complete subgraph (a clique) of G .

Decision form: Does G contain a clique of size k ?

Optimization form (Maximum Clique): Find the largest such subset S for which the induced subgraph is complete.

2. Constraints

The solution must satisfy the following constraints:

- Vertex Adjacency Constraint: Every pair of vertices selected into the candidate set must be directly connected by an edge.
- Clique Size Constraint: The selected subset must contain exactly k vertices.
- Subset Selection Constraint: Vertices must be chosen only from V , without repetition.
- Completeness Constraint: The induced subgraph on the selected vertices must be a complete graph, i.e. every vertex must be adjacent to all other vertices already chosen.
- Ordering Constraint (implementation-level): Candidates are considered in increasing index order to avoid generating the same subset more than once.

3. Effect of Constraints on Search Space

Without constraints, the number of possible k -sized subsets of an n -vertex graph is $C(n, k)$, which already grows combinatorially; if k is allowed to vary, the total number of subsets to inspect is 2^n .

The constraints substantially reduce the effective search space explored by backtracking:

- The adjacency constraint rejects a candidate vertex the moment it is found to be non-adjacent to any vertex already in the partial clique, so entire subtrees below that candidate are never generated.
- The size constraint bounds the recursion depth to exactly k , so no branch is explored beyond that depth.
- The ordering constraint (only considering vertices with index greater than the last selected vertex) eliminates duplicate permutations of the same subset, cutting the search space roughly by a factor of $k!$.
- The remaining-vertices constraint prunes a branch as soon as the number of vertices left to examine is smaller than the number still needed to reach size k .

Consequently, although the theoretical worst case remains exponential (since Clique is NP-Complete), the constraints allow backtracking to abandon the vast majority of invalid partial solutions early rather than expanding them fully.

4. Backtracking Algorithm

Input: Graph $G(V, E)$, integer k

Output: TRUE with a witness clique if a clique of size k exists in G , otherwise FALSE

```

CLIQUE-DETECT( $G, k$ )
   $n \leftarrow |V|$ 
   $currentClique \leftarrow \emptyset$ 
   $found \leftarrow FALSE$ 

  BACKTRACK( $start, currentClique$ )
    if  $|currentClique| = k$ 
       $found \leftarrow TRUE$ 
      report  $currentClique$  as a valid clique
      return
    if  $n - start < k - |currentClique|$ 
      return // not enough vertices left, prune

    for  $v = start$  to  $n - 1$ 
      if isAdjacentToAll( $v, currentClique$ )
        add  $v$  to  $currentClique$  // include the vertex
        BACKTRACK( $v + 1, currentClique$ )
        remove  $v$  from  $currentClique$  // backtrack (exclude the vertex)
      if  $found$ 
        return

  BACKTRACK( $0, \emptyset$ )
  return  $found$ 

```

The helper `isAdjacentToAll( $v, currentClique$ )` checks whether vertex v is connected by an edge to every vertex already present in $currentClique$, which is the core feasibility test applied at every node of the search tree.

5. Pruning Techniques

A. Adjacency Pruning

If a candidate vertex v is not adjacent to at least one vertex already present in the partial clique, v is rejected immediately and the branch is not expanded further — it can never lead to a valid clique.

B. Remaining-Vertices Pruning

If the number of vertices left to examine ($n - start$) is smaller than the number of additional vertices still required to reach size k , the branch is discarded, since it is mathematically impossible to complete the clique.

C. Size-Bound Pruning

As soon as the partial clique reaches size k , the search along that branch stops — there is no need to add further vertices, and (for the decision problem) the overall search can terminate.

D. Ordering / Symmetry Pruning

Candidates are always drawn from indices greater than the most recently added vertex. This avoids revisiting the same subset in a different order, removing a large amount of redundant search.

6. Example

Consider a graph with vertex set $V = \{A, B, C, D, E\}$ and the following edges:

Edge	Connects
A-B	$A \leftrightarrow B$
A-C	$A \leftrightarrow C$
B-C	$B \leftrightarrow C$

B-D	$B \leftrightarrow D$
C-E	$C \leftrightarrow E$

Suppose $k = 3$. Backtracking starts with A, extends to B (adjacent to A), then tries C: C is adjacent to both A and B, so $\{A, B, C\}$ is a valid candidate. Since $|\{A, B, C\}| = 3 = k$, and every pair (A-B, A-C, B-C) is connected, $\{A, B, C\}$ is reported as a clique of size 3.

Branches such as $\{A, B, D\}$ fail because D is not adjacent to A, so D is rejected by adjacency pruning before the branch is expanded further.

Therefore, the algorithm confirms: a clique of size 3 exists, namely $\{A, B, C\}$.

7. Why the Solution Is Correct

The algorithm systematically explores every candidate extension of the current partial clique in increasing vertex order, so no valid combination of vertices is skipped.

A branch is discarded only in two situations: when adding a vertex would violate the adjacency (completeness) constraint, or when it is provably impossible to reach size k with the vertices remaining. Neither condition can eliminate a genuine solution, since any true clique of size k must, by definition, have every pair of its vertices adjacent, and must contain at least k vertices from the remaining candidates at every step along the path that builds it.

Hence, if a clique of size k exists anywhere in G , the backtracking search is guaranteed to reach and report it; if none exists, the search exhausts all branches and correctly returns FALSE.

8. Complexity and NP-Completeness

The Clique Detection decision problem — "does G contain a clique of size k ?" — belongs to the class NP: given a candidate subset of k vertices as a certificate, verifying that every pair is adjacent can be done in polynomial time, specifically $O(k^2)$.

The problem is also NP-Hard: it can be shown to be at least as hard as other known NP-Complete problems through polynomial-time reduction (for example, from the Independent Set problem, since a clique in G corresponds exactly to an independent set in the complement graph of G).

Since Clique Detection is both in NP and NP-Hard, it is classified as NP-Complete.

In the worst case, unconstrained backtracking may need to examine on the order of $C(n, k)$ candidate subsets, which is bounded above by $O(n^k)$, and in the fully general case (k unspecified) by $O(2^n)$ subsets overall. Verification of each candidate subset costs $O(k^2)$. Pruning techniques such as adjacency checks, remaining-vertices bounds, and ordering constraints substantially reduce the practical number of branches explored, but the worst-case complexity remains exponential, consistent with the NP-Complete nature of the problem.

9. Sample Execution

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 2
1 3
2 4
Enter required clique size (k): 3

Checking for a clique of size 3 ...
Clique Found: [0, 1, 2]
Vertices 0-1, 0-2, and 1-2 are mutually adjacent.

=== Code Execution Successful ===
```

10. Conclusion

The Clique Detection Problem illustrates how constraint-based reasoning strengthens a backtracking search on combinatorial graph problems. The adjacency and size constraints ensure that only genuinely complete subgraphs of the required size are ever reported, while pruning — through adjacency checks, remaining-vertices bounds, and ordering — eliminates large parts of the search tree that cannot possibly yield a valid clique. Despite these optimizations, because Clique Detection is NP-Complete, its worst-case time complexity remains exponential, and no known polynomial-time algorithm can solve it for all graphs unless $P = NP$.